Hierarchical Planning with JSHOP2

Aleksandra Słomska
Zofia Narloch

May 3, 2026

1 Exercise 1.1: Emergency Service Logistics, Initial Release

In this exercise we recreated the Emergency Service Logistics domain from the PDDL labs, but this time using JSHOP2, which is a hierarchical planner. The main difference is that instead of just describing what actions are possible (like in PDDL), we also have to tell the planner *how* to organize those actions into higher-level tasks.

Below is an example problem we defined for JSHOP2. There are three people at different locations, and the drone needs to deliver food and medicine to them:

```
(defproblem problem p1
  (
    (drone-at drone1 depot)
    (crate-at crate1 depot)
    (crate-at crate2 depot)
    (crate-at crate3 depot)
    (empty-arm drone1 left)
    (empty-arm drone1 right)
    (content-crate food crate1)
    (content-crate food crate2)
    (content-crate medicine crate3)
    (person-at person1 loc1)
    (person-at person2 loc3)
    (person-at person3 loc2)
    (needs-content person2 food)
    (needs-content person2 medicine)
    (needs-content person3 food)
  )

  (
    (deliver-all)
  )
)
```

Here the goal is just (deliver-all) — a single high-level task. In PDDL, we would instead write out each individual goal condition explicitly, like (has-content-person person2 food) and (has-content-person person2 medicine). In JSHOP2, we let the task hierarchy figure out what needs to be done.

The planner produced the following plan:

```
(!pick-up-crate crate1 depot drone1 left)
(!drone-move depot loc3 drone1)
(!drop-crate loc3 person2 crate1 left food drone1)
(!drone-move loc3 depot drone1)
(!pick-up-crate crate3 depot drone1 right)
(!drone-move depot loc3 drone1)
(!drop-crate loc3 person2 crate3 right medicine drone1)
(!drone-move loc3 depot drone1)
(!pick-up-crate crate2 depot drone1 left)
(!drone-move depot loc2 drone1)
(!drop-crate loc2 person3 crate2 left food drone1)
--------------------

Time Used = 0.009
```

Listing 1: First problem and the outcome - three people and three boxes

The plan looks very similar to what PDDL planners produce — the individual actions (pick up, move, drop) are the same. The difference is in how the planner *found* the plan, not in what the plan looks like.

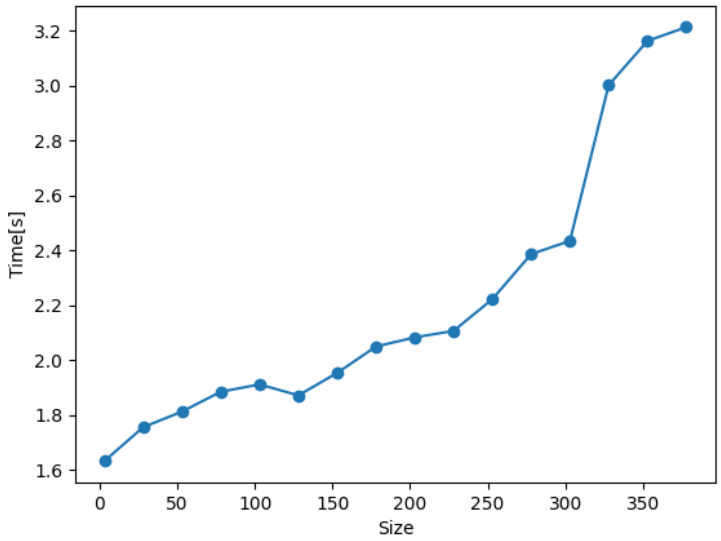

Figure 1: Size of the problem and the time required to find a solution in the tests

It is important to note that the time shown on the graph is measured by the Python script. The time reported by JSHOP2 itself was below 1 second for every single problem. The graph shows that even at size 380 the total time was around 3.2 seconds — and most of that is the Python overhead.

1.1 Comparison with Classical Planning (BFS from Exercise 1.3)

In Part 1.3 of the PDDL project, we tested several search algorithms. The best performing one was BFS (Breadth-First Search) with no heuristic. It solved a problem of size 17 in just 0.77 seconds. That was the largest problem any algorithm could solve within the 60-second time limit.

Now, JSHOP2 solved a problem of size **380** in under one second of actual planning time. That is more than 22 times larger. This is a huge difference, and it is worth understanding why.

BFS works by expanding every possible state one step at a time, level by level. In our domain, the drone has many valid moves at each step — it can pick up different crates, move to different locations, drop with different arms, and so on. The number of states grows extremely fast as the problem gets bigger. By size 17, BFS was already hitting the 60-second wall because it had to check too many combinations.

JSHOP2 does not search through states in the same way. Instead, it follows a task hierarchy that was defined in the domain file. The top-level task (deliver-all) is broken down into smaller tasks, like delivering to each person one at a time. Each of those is broken down further into concrete actions: pick up a crate, fly to the person, drop the crate.

The downside is that JSHOP2 does not guarantee an optimal plan. In example above, the drone goes to loc3 twice to deliver food and medicine separately to person2, even though it could have carried both crates at once. BFS, on the other hand, would find the shortest possible plan because it explores all options at each depth level. In this domain, every BFS plan had length 17, which is optimal.

1.2 Advantages of Hierarchical Planning

- **Scalability.** While BFS could not go past size 17, JSHOP2 handled size 380 comfortably.
- **Speed.** The planner commits to a plan quickly by following the hierarchy. There is no exhaustive search.

1.3 Disadvantages of Hierarchical Planning

- **No optimality guarantee.** As mentioned above, JSHOP2 made two separate trips to deliver to person2 even though one trip would have been enough.
- **More work upfront.** Writing the JSHOP2 domain requires much more thought than writing a PDDL domain. We need to design the task hierarchy, define methods for every situation.

2 Advanced Emergency Logistics Domain

In the second part of the exercise, we upgraded our JSHOP2 domain to handle more realistic logistics using numerical fluents and transporters. Instead of dealing with individual crates and people, the drone now manages overall quantities.

2.1 How It Works

We completely changed how the drone makes decisions by breaking deliveries down into a two-step process:

1. **Filling the Transporter:** The drone stays at the depot and continues loading boxes into the transporter until it hits maximum capacity or all needs are assigned.

2. **Dispatching:** The drone flies the transporter to the required locations, dropping off the exact amounts needed before returning.

To make the drone smart enough to handle the 11 specific edge cases from the assignment, we added custom LISP axioms. These allow the planner to evaluate the current state and actively choose the *smallest* transporter that fits the cargo, or the *largest* available if nothing fits perfectly. It also dynamically prioritizes locations with the greatest need. Finally, we introduced a `reserved` fluent to securely keep track of which boxes inside the transporter belong to which destination.

2.2 Flight Cost

The cost of moving a drone without a transporter has a base cost of 50. When carrying a transporter, the cost increases proportionally to the transporter's capacity: $\text{cost} = 50 + \frac{\text{capacity}}{10}$. For example, a transporter with capacity 100 costs 60 per flight.

2.3 Testing the Scenarios

We generated different problem files to verify all 11 required situations. The table below summarizes how each one is handled:

	Situation	Method Used
1	No transporters available	`fallback-single-box`
2	One transporter available	`use-transporter`
3	Different box types on the same transporter	`fill-transporter` loop
4	Remaining boxes delivered without transporter	`fallback-single-box`
5	Transporter smaller than need → load to max	`load-partial` branch
6	Transporter larger than need → partial load	`load-fits` branch
7	Multiple transporters fit → choose smallest	`smaller-fitting-transporter-exists`
8	All transporters too small → choose largest	`larger-transporter- exists`
9	Re-evaluate after each depot return	recursive `deliver-all`
10	Serve greatest need first	`greater-need-exists`
11	Multi-location trip without returning to depot	`dispatch-transporter`

Table 1: The 11 required situations and how they are handled in the domain.

Below is an example plan that demonstrates several of these situations at once (3, 6, 11). The drone loads both food and medicine at the depot, delivers to `loc2`, and then continues directly to `loc3` without returning:

```
(!reserve-delivery carrier1 food loc2 1.0)
(!load-transporter carrier1 food 1.0 depot drone1)
(!reserve-delivery carrier1 medicine loc2 1.0)
(!load-transporter carrier1 medicine 1.0 depot drone1)
(!reserve-delivery carrier1 food loc3 1.0)
(!load-transporter carrier1 food 1.0 depot drone1)
(!drone-move-transporter depot loc2 drone1 carrier1)
(!unload-transporter carrier1 food 1.0 loc2 drone1)
(!unload-transporter carrier1 medicine 1.0 loc2 drone1)
(!drone-move-transporter loc2 loc3 drone1 carrier1)
(!unload-transporter carrier1 food 1.0 loc3 drone1)
(!drone-move-transporter loc3 depot drone1 carrier1)
```

Listing 2: Multi-stop plan: different box types delivered across multiple locations in one trip